



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Laboratorio No. 3: Clases pt3.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Gustavo O. Zamarron De La Garza

Septiembre/2026

Tabla de contenido

I.	Objetivo	3
II.	Descripción de la practica.....	3
	Lista de materiales.....	3
III.	Desarrollo de la práctica	4
	Metodología y Arquitectura	4
	Descripción del problema y elementos clave	4
	Datos de entrada y datos de salida	4
	Diagrama de clases.....	5
	Fragmentos de código y visualización de elementos clave	6
	Clase Mascota	6
	Método registrar()	6
	Método darDiagnostico()	7
	Menú principal y método test()	7
	Reflexión y análisis crítico.....	8
	Dificultades técnicas.....	8
	Fracasos, fallas, bugs y errores	8
	Cómo se resolvió.....	8
	Conclusión	8
IV.	Bibliografía.....	8

I. Objetivo

El alumno diseñará clases para modelar el estado y comportamiento de entidades del mundo real mediante atributos privados, constructores, getters/setters y colaboración entre objetos. En esta práctica se implementó una clase Mascota que representa a un paciente de la veterinaria, y una clase principal Veterinaria que administra un arreglo de mascotas a través de un menú interactivo para registrar mascotas y asignarles diagnósticos.

II. Descripción de la practica

Haz un programa que le facilite el registro de mascotas (máximo 5) y la asignación de diagnósticos de la clínica veterinaria.

El programa debe tener el siguiente menú y realizar la operación correspondiente:

1.- Registro mascota

2.- Dar diagnóstico

3.- Salir

En registro, es crear el objeto de mascota en el arreglo.

En dar diagnóstico es buscar una mascota y cambiarle el diagnóstico, mostrar que realmente se hizo el cambio (imprimir la mascota).

Salir finaliza el programa.

Lista de materiales

- Computadora personal (PC).
- Ambiente de desarrollo (editor de texto / IDE) y compilador javac.
- JDK (Java Development Kit).

III. Desarrollo de la práctica

Metodología y Arquitectura

Descripción del problema y elementos clave

En este laboratorio, la especificación indicaba explícitamente qué clase de datos crear (Mascota) y qué comportamiento debía tener el programa principal (Veterinaria): un menú con las opciones registrar mascota, dar diagnóstico y salir, un arreglo de máximo 5 mascotas, y una búsqueda por nombre para actualizar el diagnóstico. Esto dejó como tarea principal decidir cómo repartir la responsabilidad entre Mascota (los datos de cada paciente) y Veterinaria (el menú, el arreglo y la búsqueda), y cómo estructurar un submenú de captura de datos para el registro.

Los elementos clave del diseño son los siguientes:

- Mascota se implementó como una clase de datos independiente, con sus cinco atributos privados (nombre, edad, tipo, raza, diagnostico), un constructor con parámetros, un constructor vacío que delega en el anterior con valores por defecto, y sus setters y getters.
- Veterinaria es la clase que contiene el método main y no está pensada para instanciarse: declara como atributos estáticos un arreglo Mascota[5], un contador de mascotas realmente registradas y un Scanner compartido, de modo que todos los métodos estáticos puedan usarlos sin recibirlos como parámetro.
- El registro se organizó como un submenú propio (elegir qué campo capturar y regresar cuando se terminó), en vez de pedir los cinco datos uno tras otro sin posibilidad de corregir o saltar alguno.

Datos de entrada y datos de salida

Datos de entrada

- Menú principal: número entero para elegir la opción (1 Registro mascota, 2 Dar diagnóstico, 3 Salir).
- Submenú de registro: número entero para elegir el campo a capturar (1 Nombre, 2 Edad, 3 Tipo, 4 Raza, 5 Diagnóstico, 0 Regresar).
- Captura de campos del registro: cadenas de texto para nombre, tipo, raza y diagnóstico, y un valor entero para la edad.
- Dar diagnóstico: cadena de texto con el nombre de la mascota a buscar, y cadena de texto con el nuevo diagnóstico a asignar.

Datos de salida

- Mensajes de confirmación de captura (por ejemplo, "Captura correcta: Firulais").
- Impresión del diagnóstico actual y del diagnóstico ya actualizado de la mascota encontrada, o "Mascota no encontrada." si no existe.

- Mensajes de navegación del menú (opción no encontrada, ya se alcanzó el máximo de mascotas registradas, saliendo del programa).

Diagrama de clases

El siguiente diagrama, elaborado en HTML/CSS, muestra las dos clases del programa: Mascota, la clase de datos de cada paciente, y Veterinaria, que la agrega (0..5) a través de un arreglo de tamaño fijo y la usa para registrar y consultar mascotas. Veterinaria no hereda de Mascota ni la compone de forma exclusiva más allá de su propio arreglo, por lo que la relación se modela como agregación.

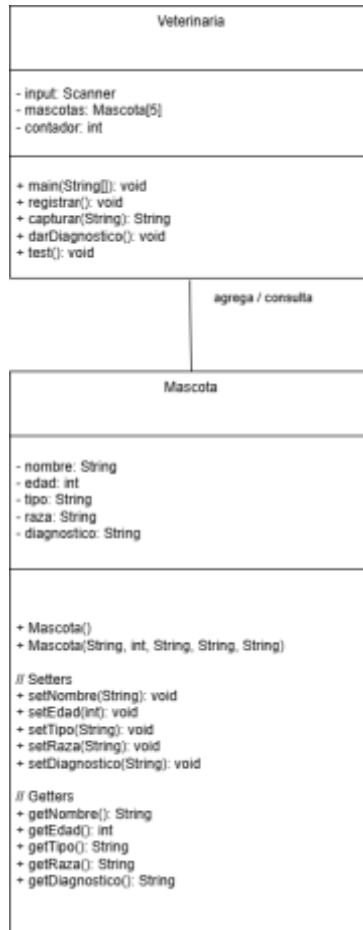


Ilustración 1: Diagrama de clases — Mascota y Veterinaria.

Fragmentos de código y visualización de elementos clave

Clase Mascota

```
public Mascota(String nombre, int edad, String tipo, String raza, String diagnostico) {
    this.nombre = nombre;
    this.edad = edad;
    this.tipo = tipo;
    this.raza = raza;
    this.diagnostico = diagnostico;
}
```

```
}

public Mascota() {
    this("", 0, "", "", "");
}
```

Diseñé dos constructores: uno con los cinco parámetros que inicializa todos los atributos directamente, y uno vacío que delega en el primero (this(...)) con valores por defecto, para que Veterinaria pueda crear un objeto “vacío” en el arreglo antes de capturar sus datos con los setters, sin duplicar la lógica de inicialización.

```
Opcion (menu principal): 1
Opcion (submenu registro): 1
Favor de ingresar nombre: Firulais
Captura correcta: Firulais
```

Método registrar()

```
if (contador >= mascotas.length) {
    System.out.println("Ya se alcanzo el maximo de mascotas registradas.");
    return;
}
```

```
if (mascotas[contador] == null) {
    mascotas[contador] = new Mascota();
}
// ...submenu: 1 Nombre, 2 Edad, 3 Tipo, 4 Raza, 5 Diagnostico...
case 1: mascotas[contador].setNombre(capturar("nombre")); break;
// ...
contador++;
```

registrar() primero verifica que no se haya alcanzado el máximo de 5 mascotas y crea el objeto Mascota vacío en el arreglo solo la primera vez que se entra a esa posición; el submenú permite elegir libremente qué campo capturar (usando los setters de Mascota) y solo incrementa contador al presionar 0 (Regresar), para no contar una mascota que quedó a medio capturar.

```
Opcion (submenu registro): 2
Favor de ingresar edad: 3
Captura correcta: 3
Opcion (submenu registro): 0
Saliendo...
```

Método darDiagnostico()

```
for (int i = 0; i < contador; i++) {
    if (mascotas[i].getNombre().equalsIgnoreCase(nombre)) {
        System.out.printf("Diagnostico actual: %s\n", mascotas[i].getDiagnostico());
        mascotas[i].setDiagnostico(capturar("nuevo diagnostico"));
        encontrada = true;
        break;
    }
}
```

darDiagnostico() busca de forma lineal en mascotas[0..contador) comparando el nombre con equalsIgnoreCase() para que la búsqueda no distinga mayúsculas/minúsculas; al encontrar la mascota imprime su diagnóstico actual antes de sobrescribirlo, para poder comprobar en consola que el cambio realmente se hizo, tal como lo pedía el enunciado.

```
Nombre de la mascota: Firulais
Diagnostico actual: Saludable
Favor de ingresar nuevo diagnostico: Fractura leve
Captura correcta: Fractura leve
```

Menú principal y método test()

```
public static void main(String[] args) {
    int opcion;
    do {
        // ...imprime el menu...
        opcion = input.nextInt();
        input.nextLine();
        switch (opcion) {
            case 1: registrar(); break;
            case 2: darDiagnostico(); break;
            case 3: System.out.println("Saliendo..."); break;
            case 4: test(); break;
            default: System.out.println("No se encontro la opcion");
        }
    } while (opcion != 3);
}
```

El menú principal de Veterinaria funciona igual que en las prácticas anteriores: un ciclo do-while con un switch que dirige a cada operación, y una opción oculta (test(), no impresa en el menú pero accesible tecleando 4) que registra tres mascotas con datos fijos y las imprime en una tabla con printf, lo que me sirvió para probar rápidamente el registro y el formato de salida sin capturar todo manualmente cada vez.

```
Opcion: 4
|Nombre  Edad  Tipo    Raza      Diagnostico
Firulais  3     Canino   Labrador  Fractura leve
Peneko    18     Canino   Shitzu    Se murio
Roma      10     Canino   Shitzu    no mira
Lucas     2      Canino   Shitzu    Esta malito
```

Reflexión y análisis crítico

Dificultades técnicas

La dificultad técnica más importante fue decidir cómo estructurar el submenú de registro para que permitiera capturar los cinco campos de una mascota en cualquier orden (o corregir uno ya capturado) sin obligar a repetir todo el proceso desde cero, en vez de un único constructor capturado de corrido. Otra dificultad fue la búsqueda de la mascota por nombre en `darDiagnostico()`, ya que el usuario puede escribir el nombre con mayúsculas o minúsculas distintas a como se registró.

Fracasos, fallas, bugs y errores

El primer intento de comparar nombres en `darDiagnostico()` usaba `equals()` en vez de `equalsIgnoreCase()`, por lo que una mascota registrada como “Firulais” no se encontraba si el usuario buscaba “firulais”. También tuve, igual que en prácticas anteriores, el problema de mezclar `input.nextInt()` con `input.nextLine()` en el mismo Scanner: al leer la opción del menú con `nextInt()` y luego intentar leer un nombre con `nextLine()`, la segunda lectura tomaba el salto de línea sobrante en vez del texto capturado.

Cómo se resolvió

El problema de la búsqueda se resolvió cambiando `equals()` por `equalsIgnoreCase()` en la comparación de nombres. El problema del Scanner se resolvió agregando `input.nextLine()` justo después de cada `input.nextInt()`, tanto en el menú principal como en los submenús, para limpiar el salto de línea pendiente antes de la siguiente captura de texto.

Conclusión

Esta práctica ayudó a reforzar el diseño de una clase de datos simple (Mascota) colaborando con una clase de control (Veterinaria) que administra un arreglo de tamaño fijo, practicando además la búsqueda lineal por un atributo de texto y el manejo cuidadoso de un Scanner compartido entre lecturas de enteros y de cadenas.

IV. Bibliografía

- [1] Oracle Corporation. (2024). Java SE Documentation. Recuperado de <https://docs.oracle.com/javase/>
- [2] Deitel, P. J., & Deitel, H. M. (2016). Java: How to Program (10th ed.). Pearson Education.